

Scyther Tool: Verifying of security protocols

Course 1

Nour EL MADHOUN

nour.el-madhoun@lip6.fr

Outline

- Cryptography: overview
- Introduction to the Scyther tool

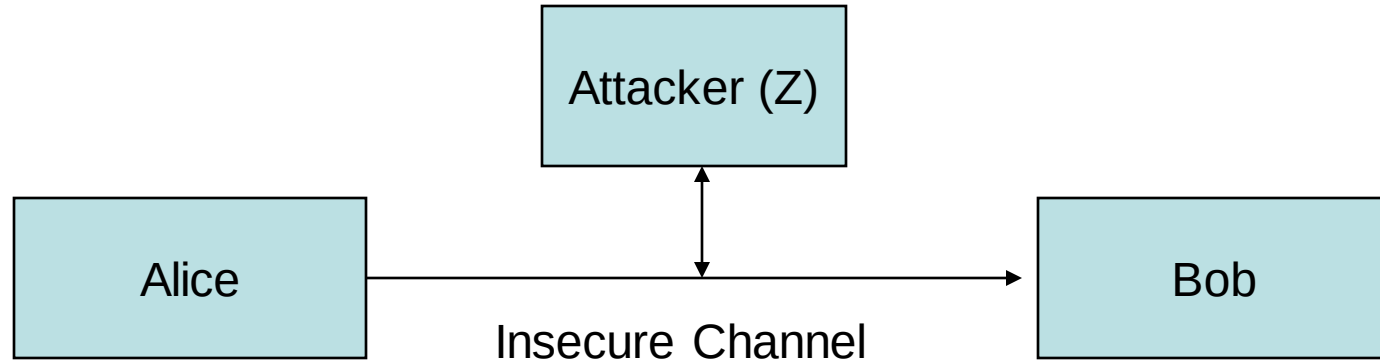
What is Cryptography ?

- Cryptography = the science (art) of encryption
- Cryptanalysis = the science (art) of breaking encryption
- Cryptology = cryptography + cryptanalysis

Security Attacks

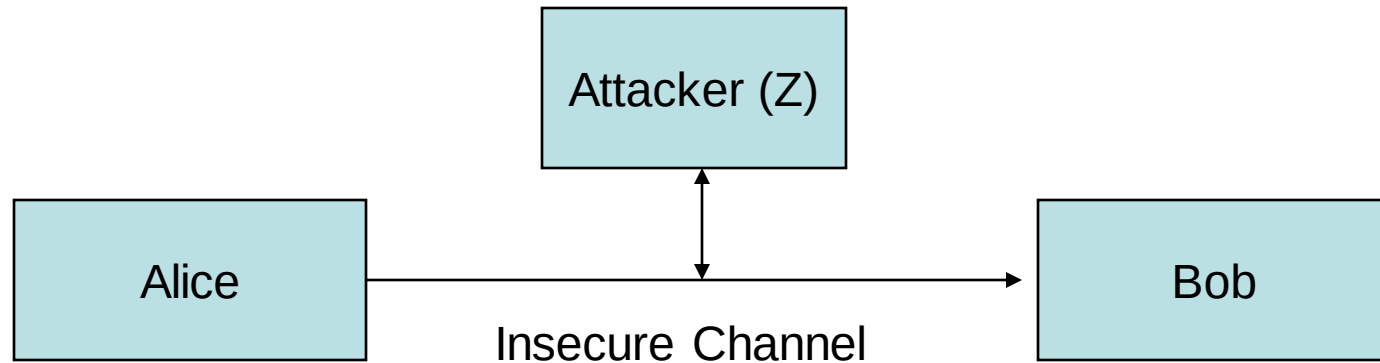
- Passive attacks
 - *Obtain* message contents
 - *Monitoring* traffic flows
- Active attacks
 - *Replay* previous messages
 - *Modify* messages in transit
 - *Add, delete* messages
 - *Denial of service*

Cryptography Goals (Security Properties)



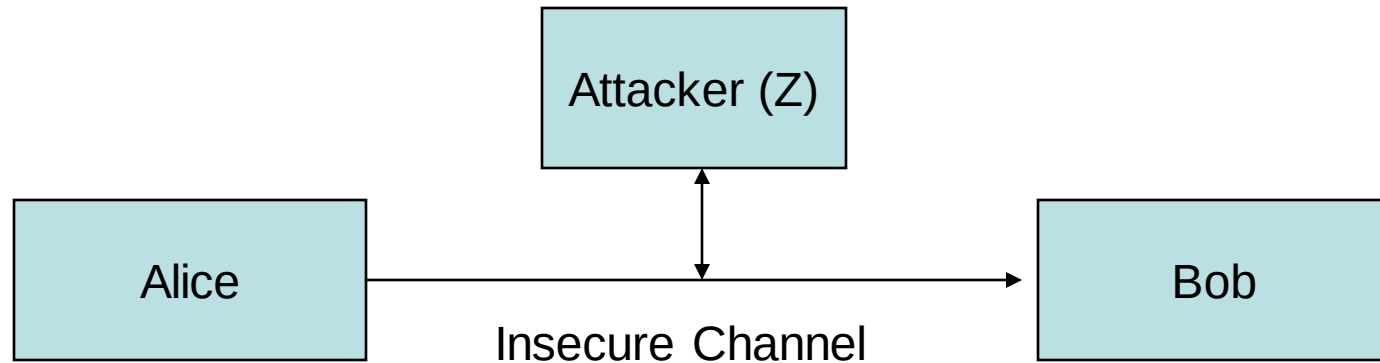
- Confidentiality (Secrecy):
 - ✓ Prevent Z from *intercepting and read* the message content
 - ✓ Only the Alice and Bob should *be able to understand* the contents of the transmitted message

Cryptography Goals (Security Properties)



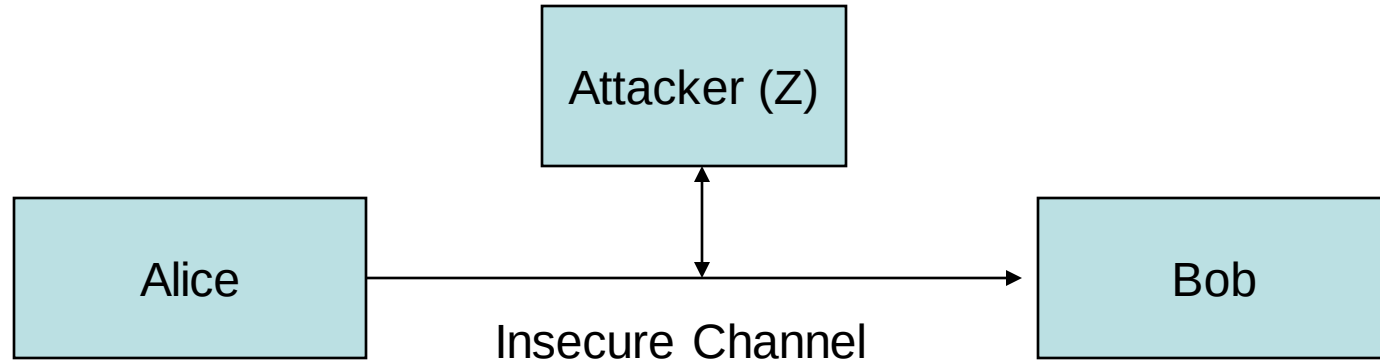
- Authentication:
 - ✓ Prevent Z from *impersonating Alice Or Bob*
 - ✓ Both *Alice and Bob need to confirm the identity* of other party involved in the communication
 - ✓ Alice *must authenticate himself* to Bob
 - ✓ Bob *must authenticate himself* to Alice

Cryptography Goals (Security Properties)



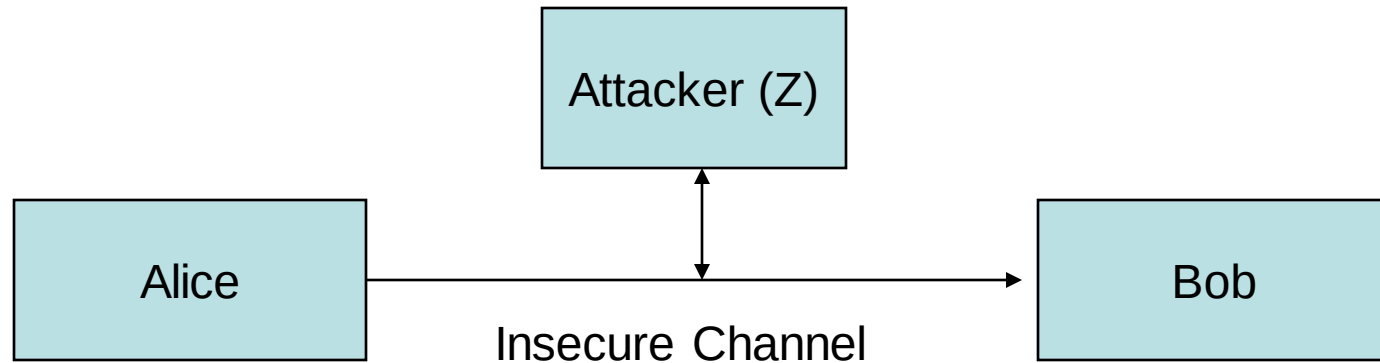
- Data Integrity:
 - ✓ Prevent Z from *modifying* the message content
 - ✓ The content of *their communication is not altered*, either maliciously or by accident, in transmission

Cryptography Goals (Security Properties)



- Non-repudiation:
 - ✓ An entity (Alice or Bob) *is prevented from denying* its previous commitments or actions

Cryptography Goals (*Security Properties*)

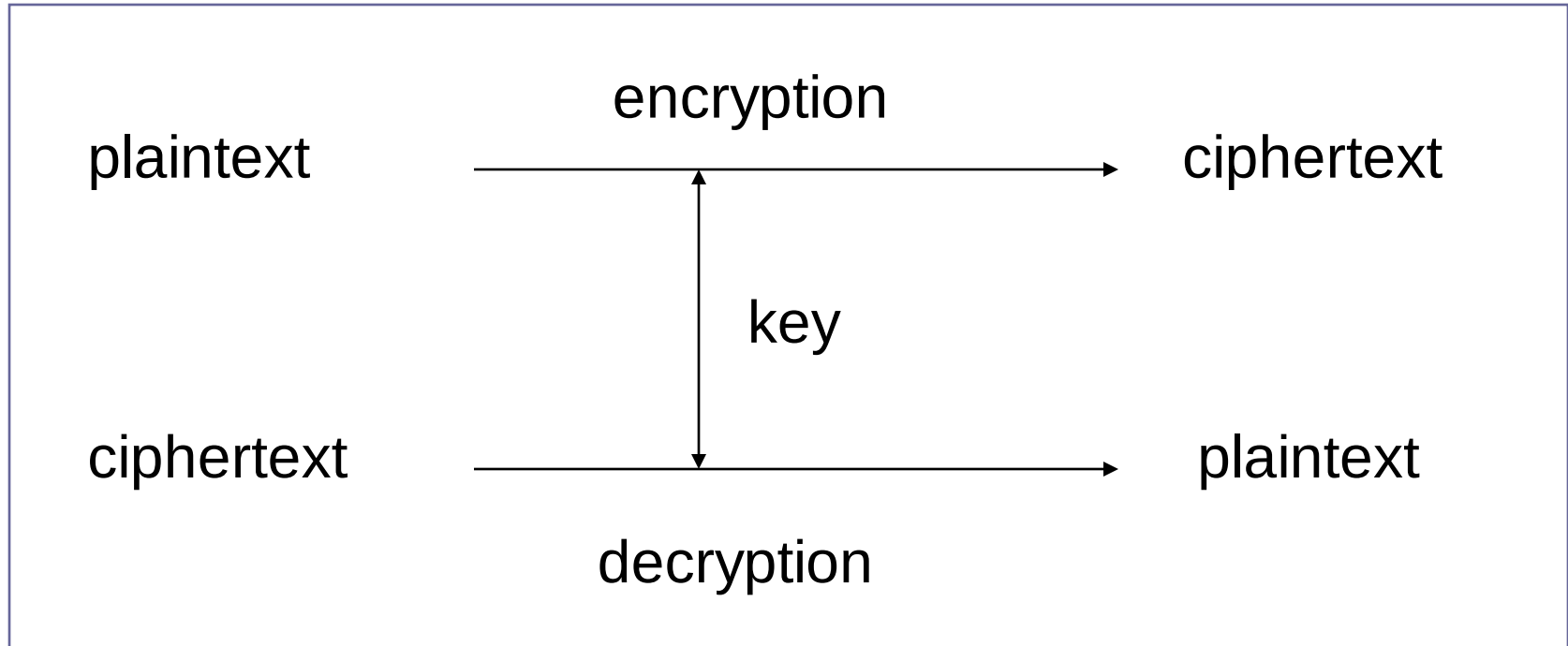


*How **to ensure these security properties** during a communication between Alice & Bob ?*

Cryptographic Functions

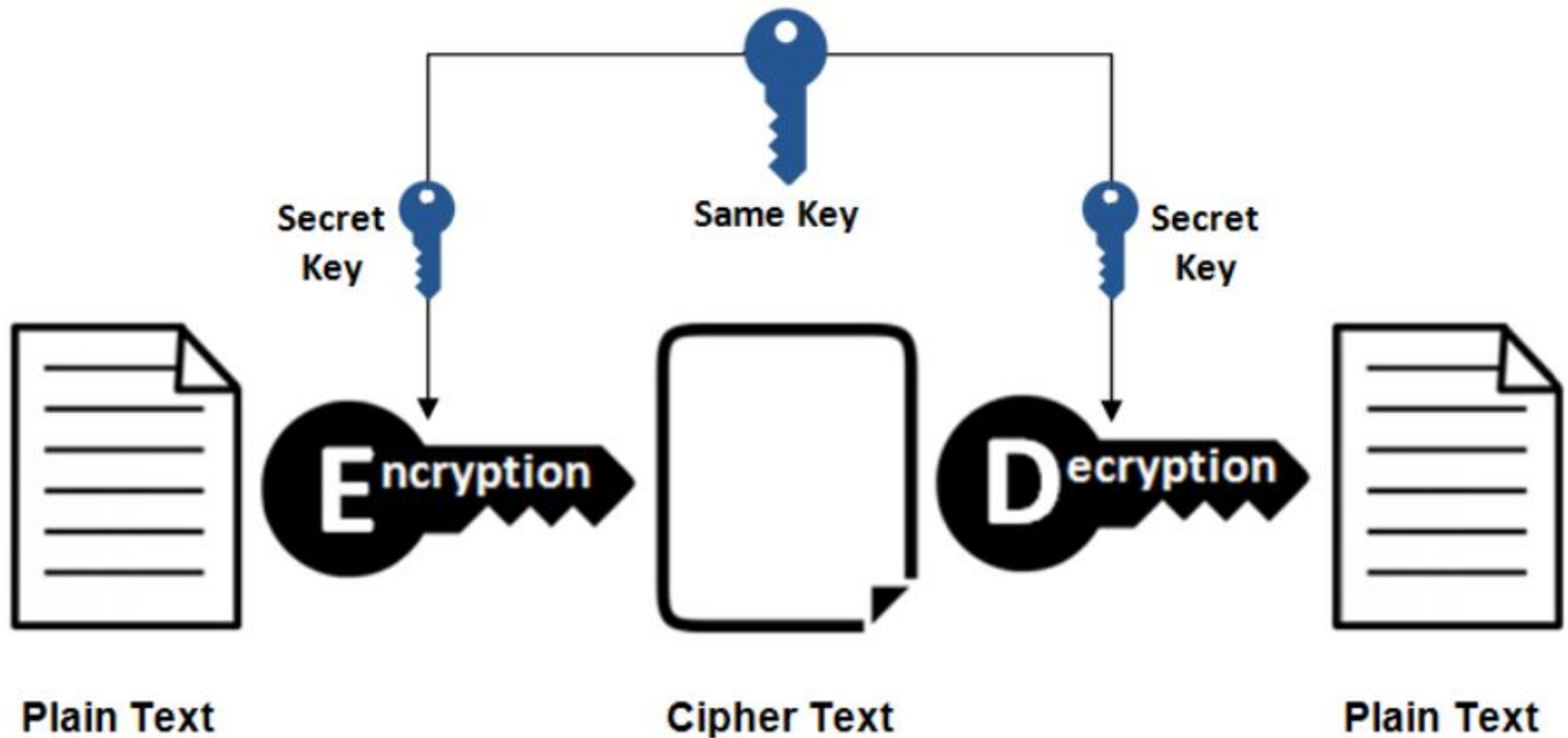
- Secret key functions (Symmetric Cryptography)
- Public key functions (Asymmetric Cryptography)
- Hash functions

Symmetric cryptography



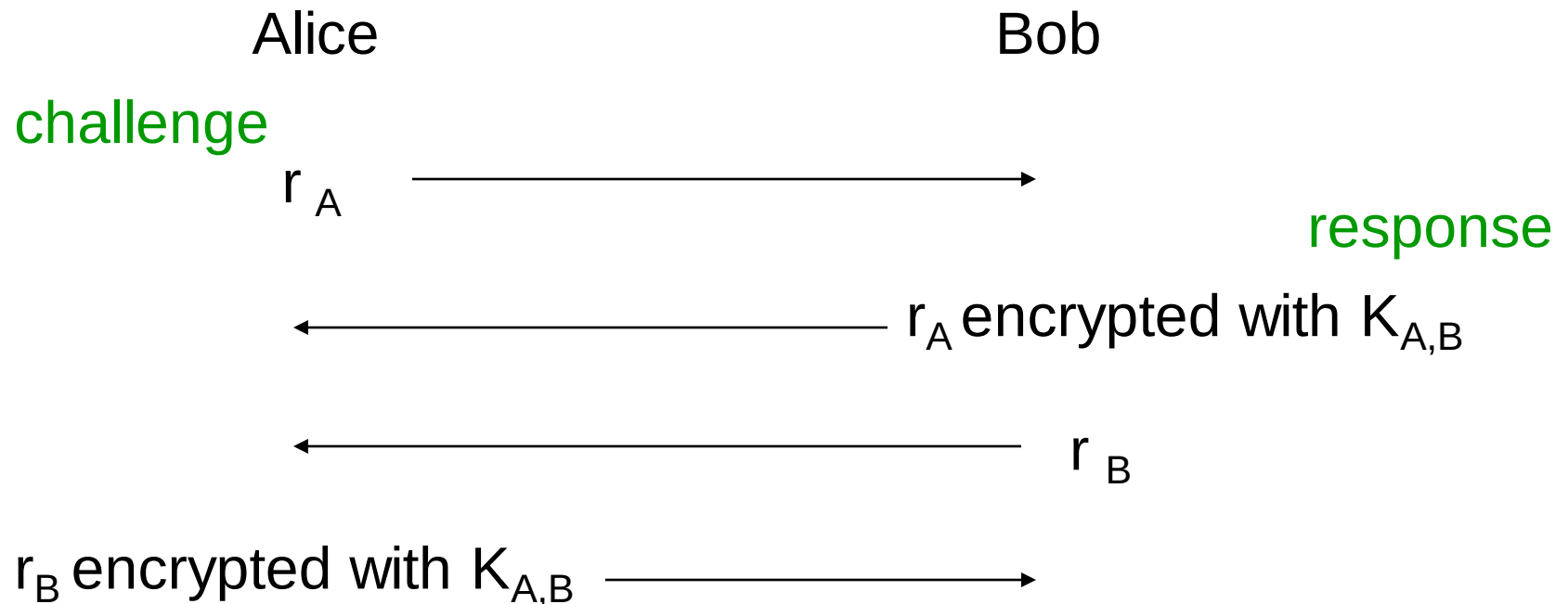
- Using a *single key for encryption/decryption*
- The *plaintext and the ciphertext* having the *same size*
- Also called *symmetric key cryptography*

Symmetric cryptography

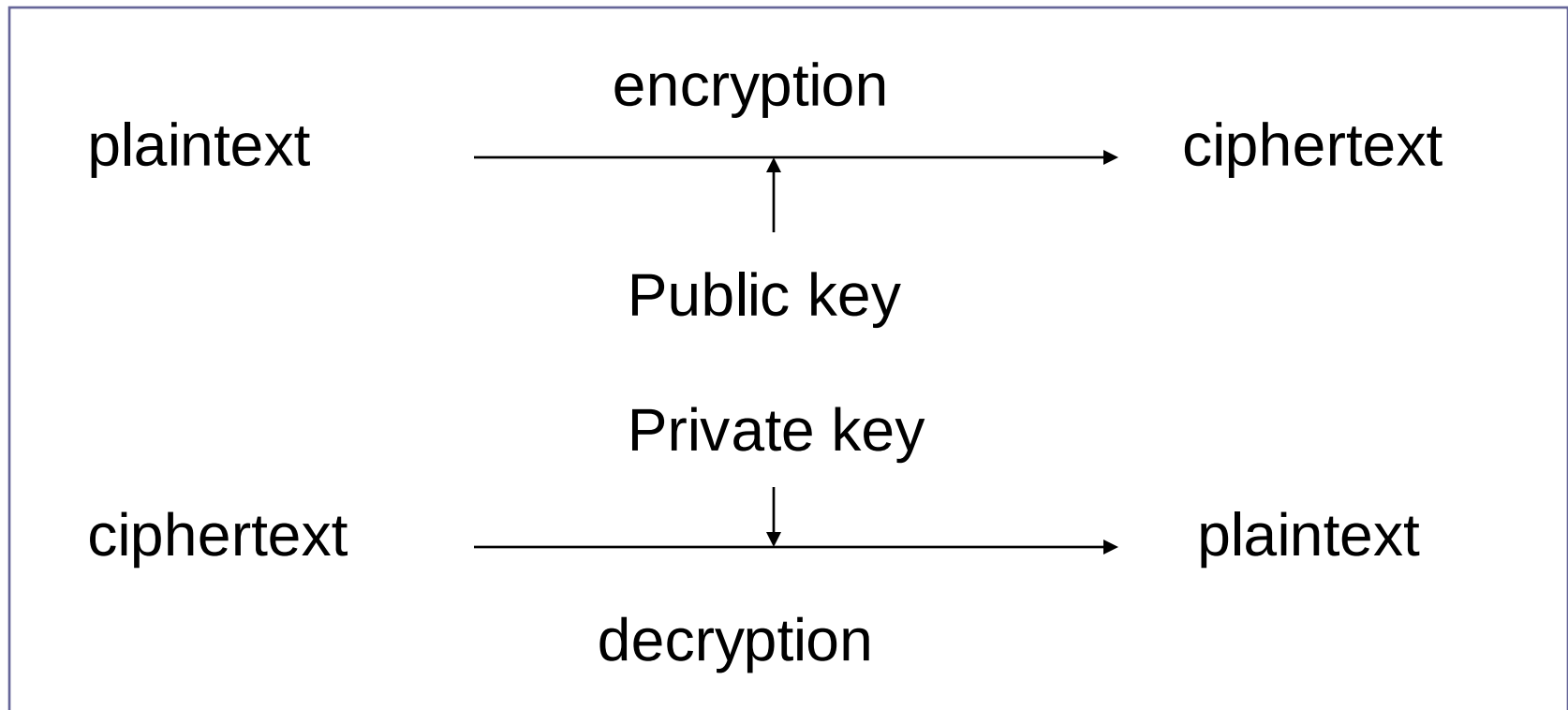


Symmetric cryptography

- Confidentiality: Prevent attackers from eavesdropping, only the entities knowing the key can decrypt it
- Authentication: Alice proves to Bob that she knows the Key

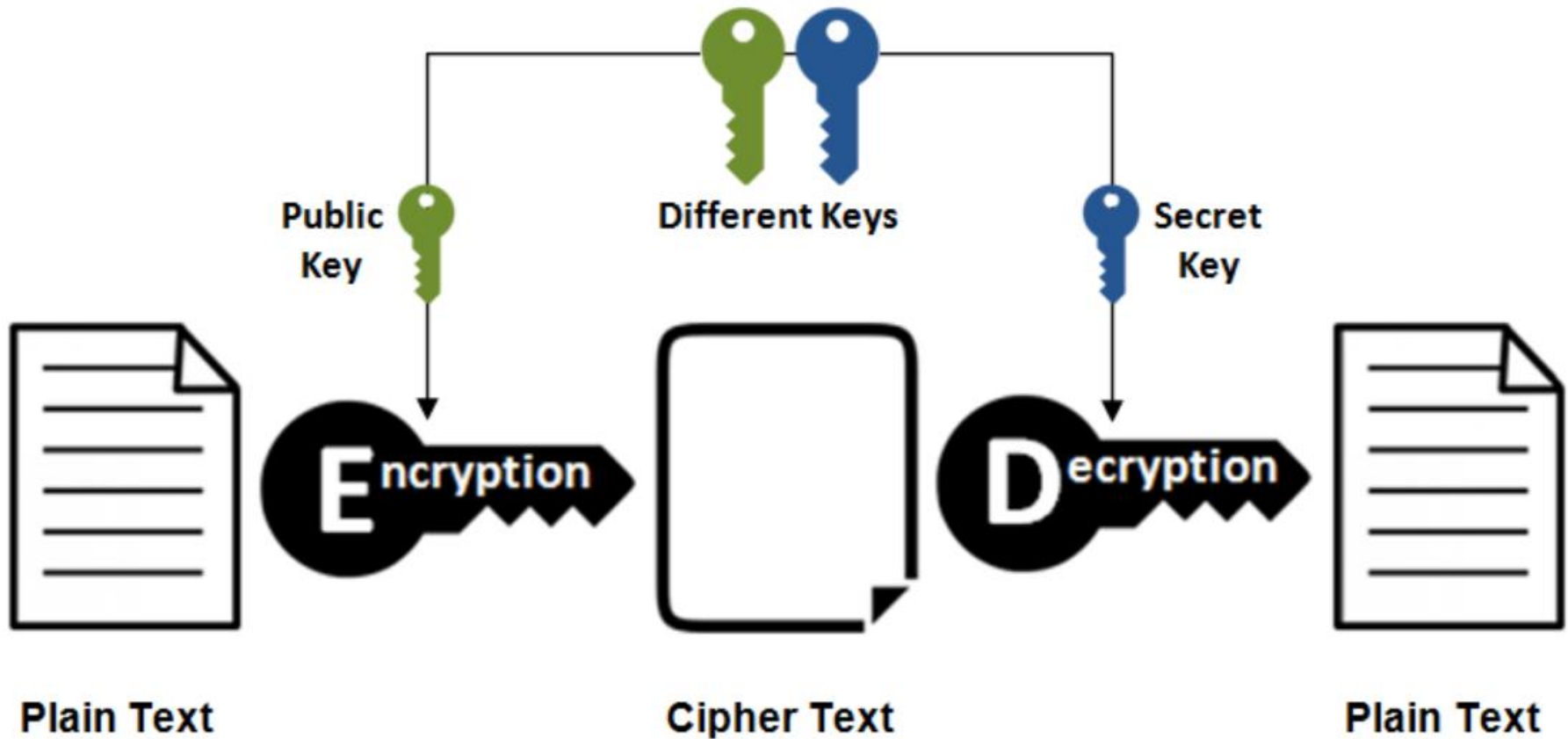


Asymmetric cryptography



- Each individual has two keys
 - a private key (**d**): need not be reveal to anyone
 - a public key (**e**): preferably known to the entire world

Asymmetric cryptography

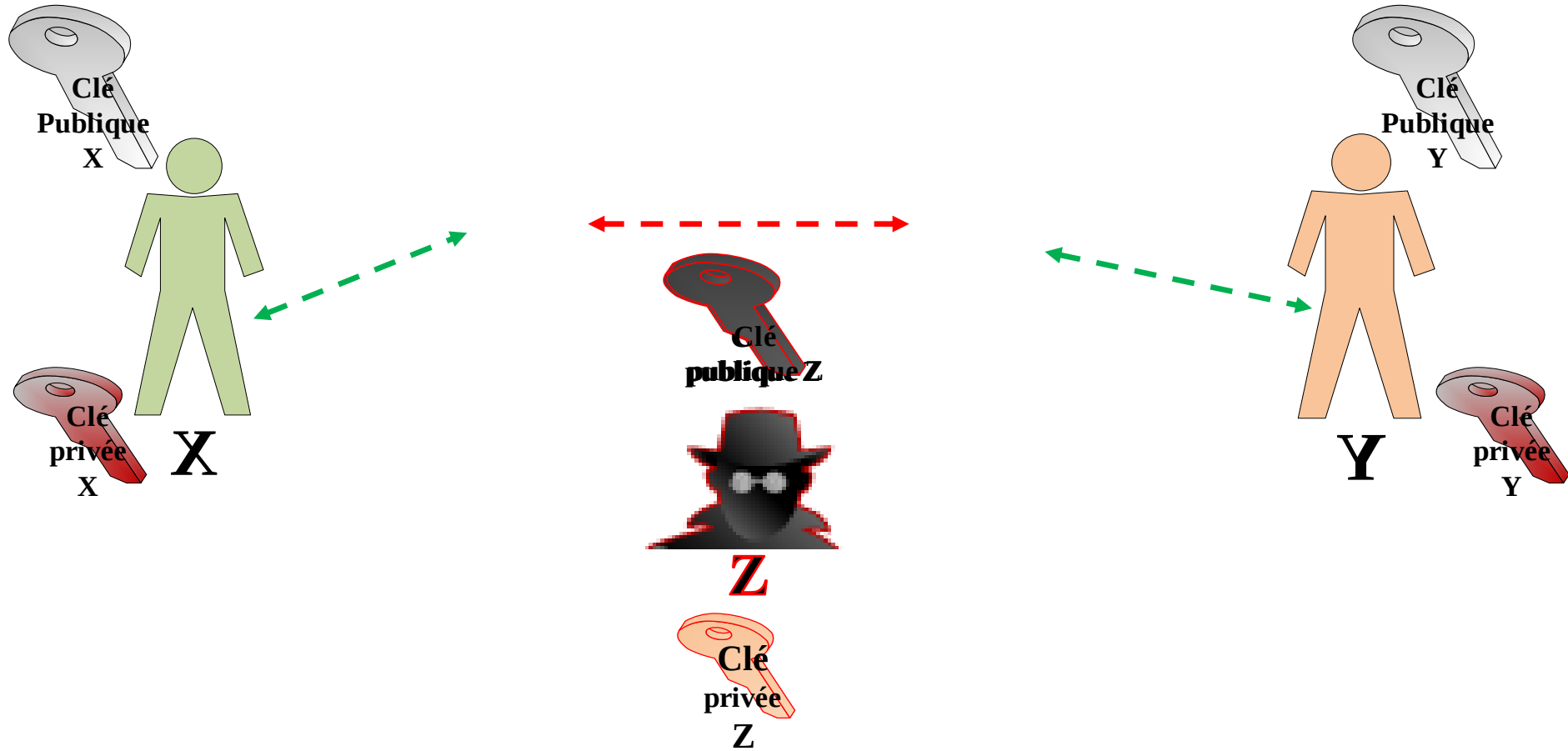


It must not be possible to compute the private key from the public key

Asymmetric cryptography

- Confidentiality: Nobody else can decrypt it (not knowing the private key of the data source)
- Authentication: How it is ensured ? If the key of encryption is public?
 - *The public key is certified by a Certification Authority (CA)*
 - *The public key is obtained from an electronic certificate*

Asymmetric cryptography

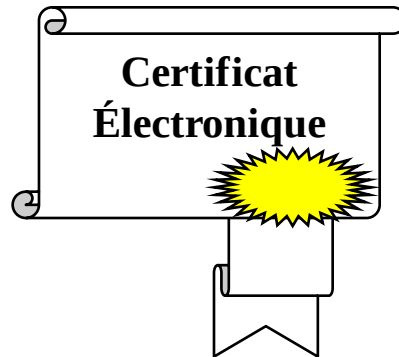


Asymmetric cryptography

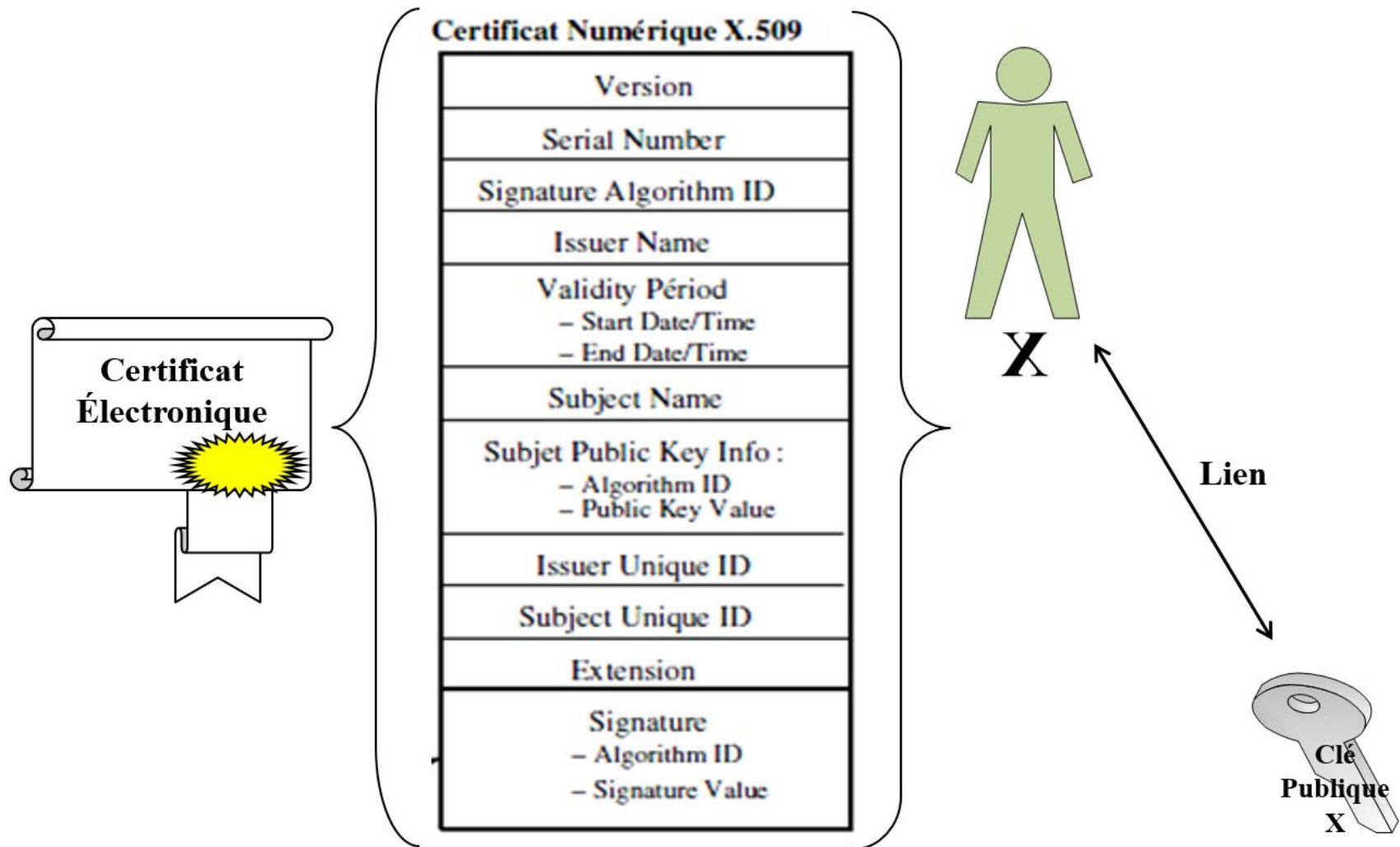
**Comment garantir l'identité lié
à la clé publique ?**



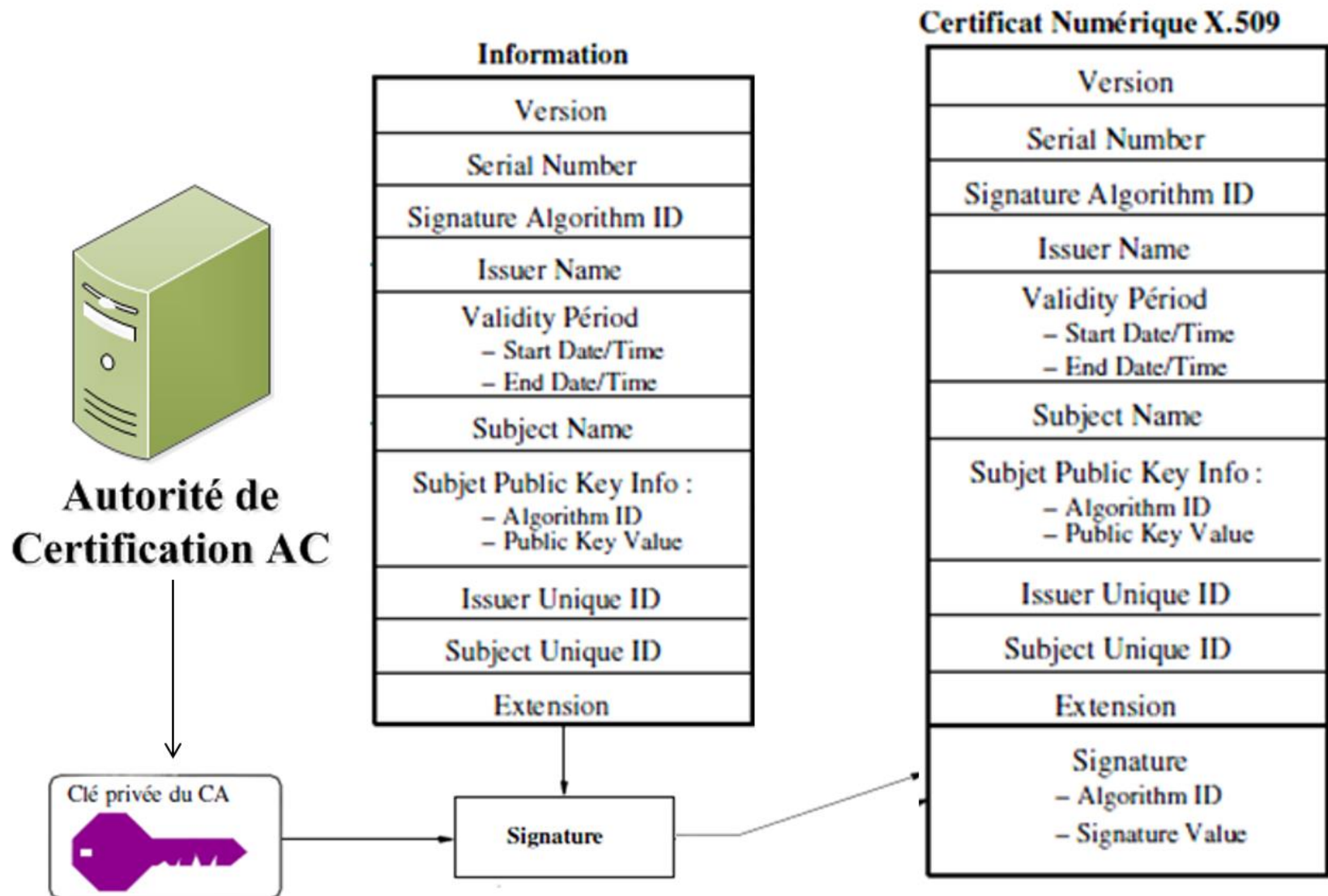
Solution



Asymmetric cryptography

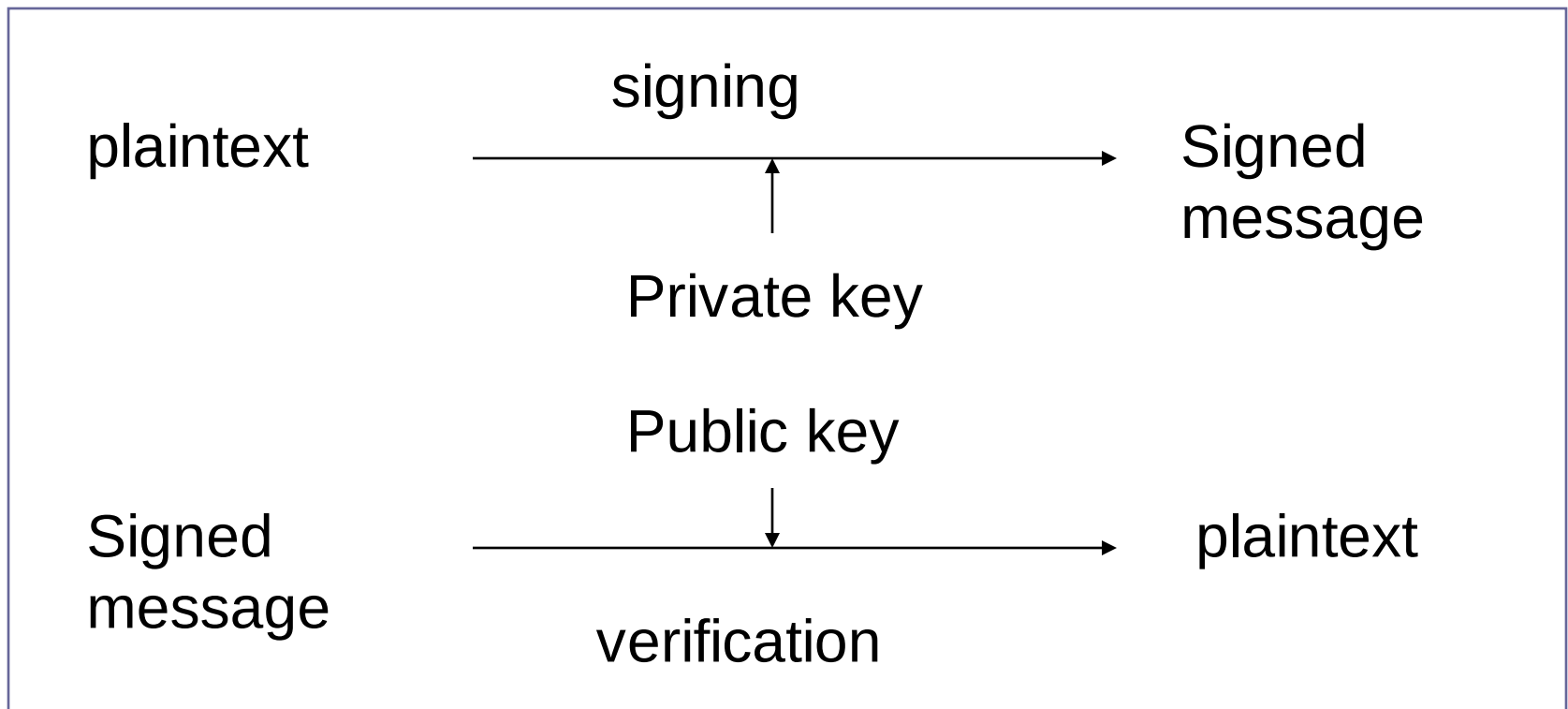


Asymmetric cryptography



Asymmetric cryptography

- Digital Signatures
 - Proving that a message *is generated by a particular individual*
 - Non-repudiation: the signing individual *can not be denied, because only him/her knows the private key*



Hash Functions



A *mathematical transformation* that takes a message of *arbitrary length* and *computes it a fixed-length* (short) number

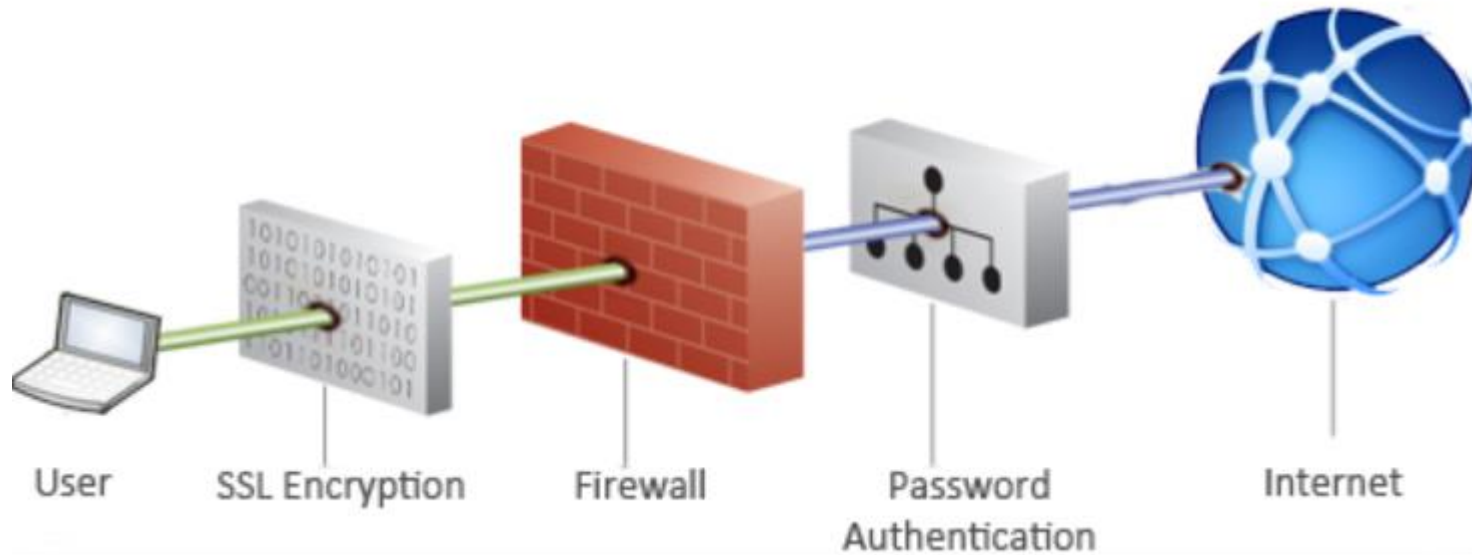
Hash Functions

- Let the hash of a message m be $h(m)$
- For any m , it is relatively easy to compute $h(m)$
- It is impossible to *find m from $h(m)$*
- It is computationally *infeasible to find two values that hash to the same thing*

Hash functions ensures message integrity

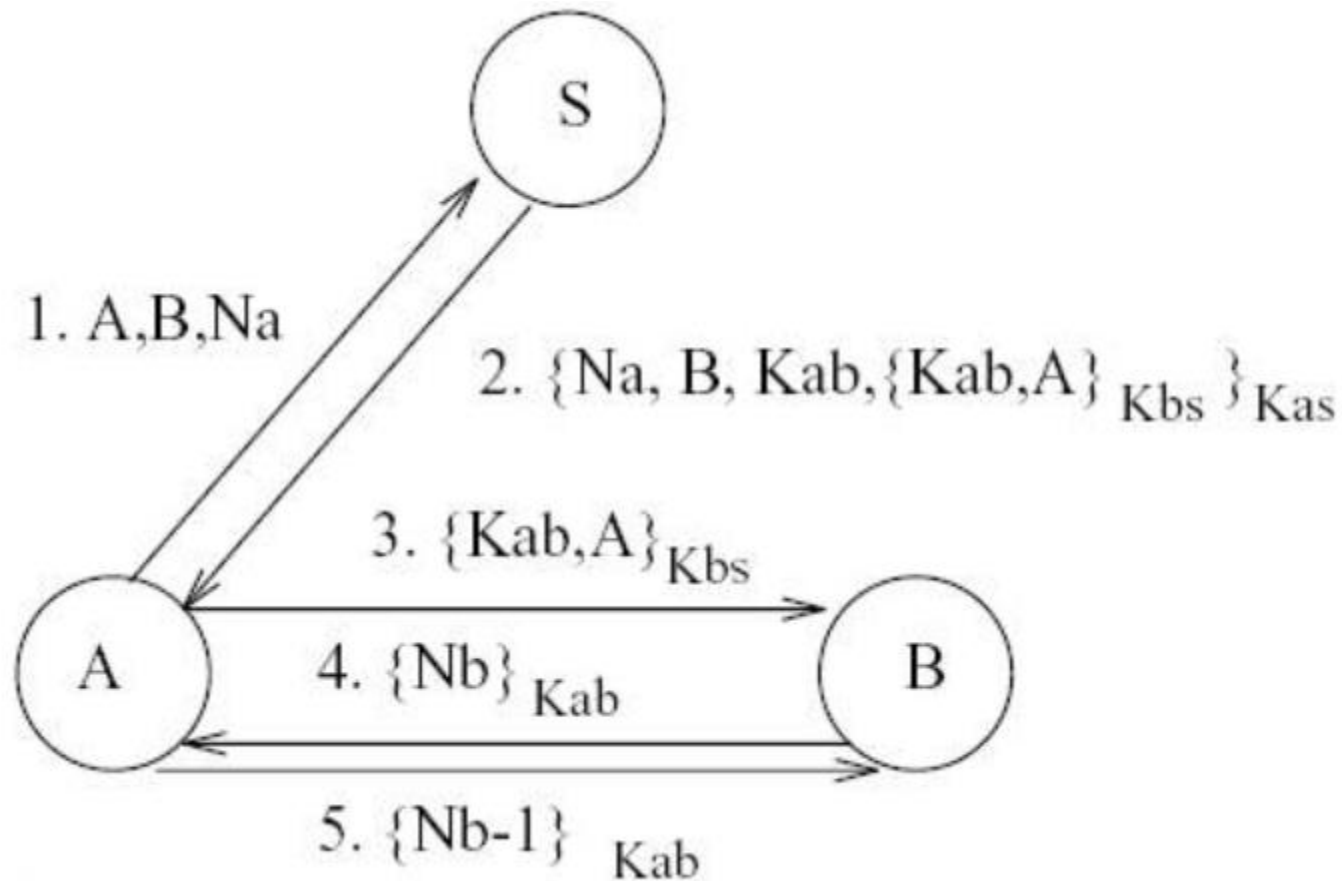
What is a security protocol ?

- Security protocol = a set of cryptoprimitives exchanged between the communication actors



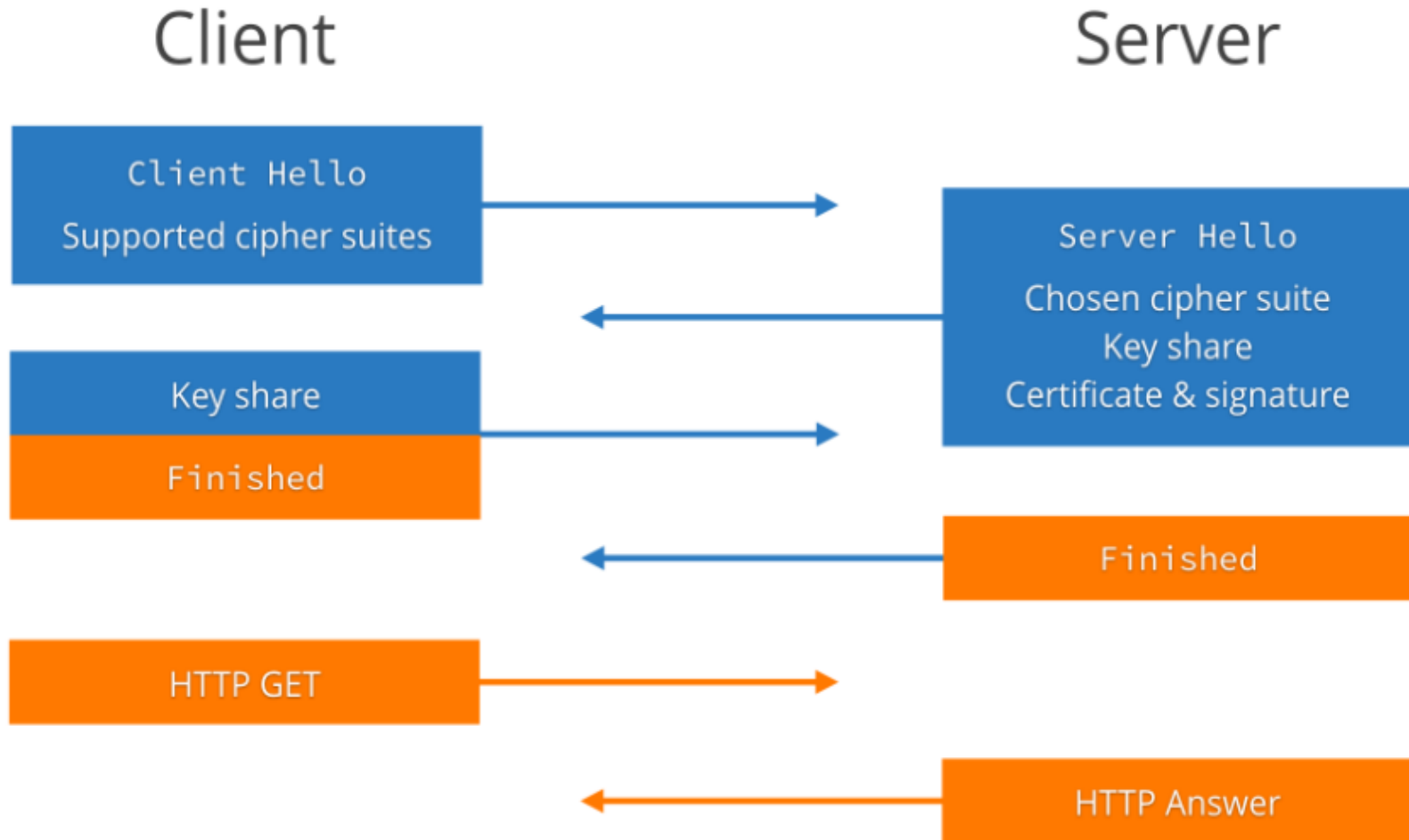
What is a security protocol ?

Exemple 1:



What is a security protocol ?

Exemple 2: TLS (overview) (next course)



Scyther Tool

- Automatic *verification* of security protocols
- *Verify the correctness of the security protocol* written in Scyther
- Analysis of security protocols to *identify potential attacks and vulnerabilities*
- Able to *detect several possible attacks*
- Generate a *graph for each attack found corresponding to the mentioned claim*

Scyther Tool

- Language used to write protocols in Scyther is:
Security Protocol Description Language (SPDL)
- *Each actor* of the security protocol is written *in a role with SPDL*
- The targeted security properties *are verified thanks to the Scyther claims*

Scyther Tool

- Language used to write protocols in Scyther is:
Security Protocol Description Language (SPDL)
- *Each actor* of the security protocol is written *in a role with SPDL*
- The targeted security properties *are verified thanks to the Scyther claims*

Scyther Tool

```
hashfunction H;  
usertype text;
```

```
protocol securityProtocol(X,Y,AS)  
{
```

```
  role X  
  {
```

```
    fresh RX: Nonce;  
    var RY: Nonce;  
    fresh dataX: text;  
    var dataY: text;
```

```
    send_1(X,Y,RX);  
    recv_2(Y,X,{RX, RY, dataY}pk(X));  
  
    send_3(X,Y,{RY, dataX, H(dataX)}pk(Y));
```

```
    .....
```

```
    claim_x1(X, Nisynch);  
    claim_y2(X, Niagree);  
    claim_x3(X, Alive);  
    claim_x4(X, Weakagree);  
    claim_x5(X, Secret, dataX);
```

```
  }
```

```
  role Y  
  {
```

```
    var RX: Nonce;  
    fresh RY: Nonce;  
    fresh dataY: text;  
    var dataX: text;
```

```
    recv_1(X,Y,RX);  
    send_2(Y,X,{RX, RY, dataY}pk(X));  
  
    recv_3(X,Y,{RY, dataX, H(dataX)}pk(Y));
```

```
    .....
```

Scyther Tool

What are Scyther claims ? (formal definitions)

✓ For authentication & non-repudiation between A and B:

Nisynch: Non-injective synchronization

Niagree: Non-injective agreement

Alive: Aliveness

Weakagree: Weak agreement

Scyther Tool

What are Scyther claims ? (formal definitions)

- ✓ For authentication & non-repudiation between A and B:

Alive: Aliveness

- *We say that a protocol guarantees to an initiator A aliveness of an agent B if, whenever A (acting as initiator) completes a run of the protocol, apparently with responder B, then B has previously been running the protocol*

Scyther Tool

What are Scyther claims ? (formal definitions)

- ✓ For authentication & non-repudiation between A and B:

Weakagree: Weak agreement

- *We say that a protocol guarantees to an initiator A weak agreement with another agent B if, whenever A (acting as initiator) completes a run of the protocol, apparently with responder B, then B has previously been running the protocol, apparently with A. Note that B may not necessarily have been acting as responder*

Scyther Tool

What are Scyther claims ? (formal definitions)

- ✓ For authentication & non-repudiation between A and B:

Niagree: Non-injective agreement

- We say that a protocol:
 - *guarantees to an initiator A non-injective agreement* with a responder B on *a set of data items ds* (where *ds* is a set of free variables appearing in the protocol description)
 - *if, whenever A (acting as initiator) completes a run of the protocol, apparently with responder B,*
 - *then B has previously been running the protocol, apparently with A, and B was acting as responder in his run,*
 - *and the two agents agreed on the data values corresponding to all the variables in ds*

Scyther Tool

What are Scyther claims ? (formal definitions)

- ✓ For authentication & non-repudiation between A and B:

Nisynch: Non-injective synchronization

- Ensures that messages *are transmitted exactly* as prescribed by the protocol.
- That is to say that :
 - whenever *A (initiator) completes running the protocol with B (responder)*,
 - and *B has been running the protocol with A*,
 - then, *all messages are received exactly as they were sent, in the exact order described by the protocol*
- It is a *Strong Authentication*

Scyther Tool

What are Scyther claims ? (formal definitions)

✓ For Confidentially

Secret

Scyther Tool

```
/*  
 * Needham-Schroeder-Lowe protocol,  
 * broken version (wrong role name in first message)  
 */
```

```
// The protocol description
```

```
protocol nsl3-broken(I,R)
```

```
{
```

```
  role I
```

```
  {
```

```
    fresh ni: Nonce;  
    var nr: Nonce;
```

```
    send_1(I,R, {ni,R}pk(R) );  
    recv_2(R,I, {ni,nr,R}pk(I) );  
    send_3(I,R, {nr}pk(R) );
```

```
    claim_i1(I,Secret,ni);  
    claim_i2(I,Secret,nr);  
    claim_i3(I,Niagree);  
    claim_i4(I,Nisynch);
```

```
  }
```

```
  role R
```

```
  {
```

```
    var ni: Nonce;  
    fresh nr: Nonce;
```

```
    recv_1(I,R, {ni,R}pk(R) );  
    send_2(R,I, {ni,nr,R}pk(I) );  
    recv_3(I,R, {nr}pk(R) );
```

```
    claim_r1(R,Secret,ni);  
    claim_r2(R,Secret,nr);  
    claim_r3(R,Niagree);  
    claim_r4(R,Nisynch);
```

```
  }
```

```
}
```

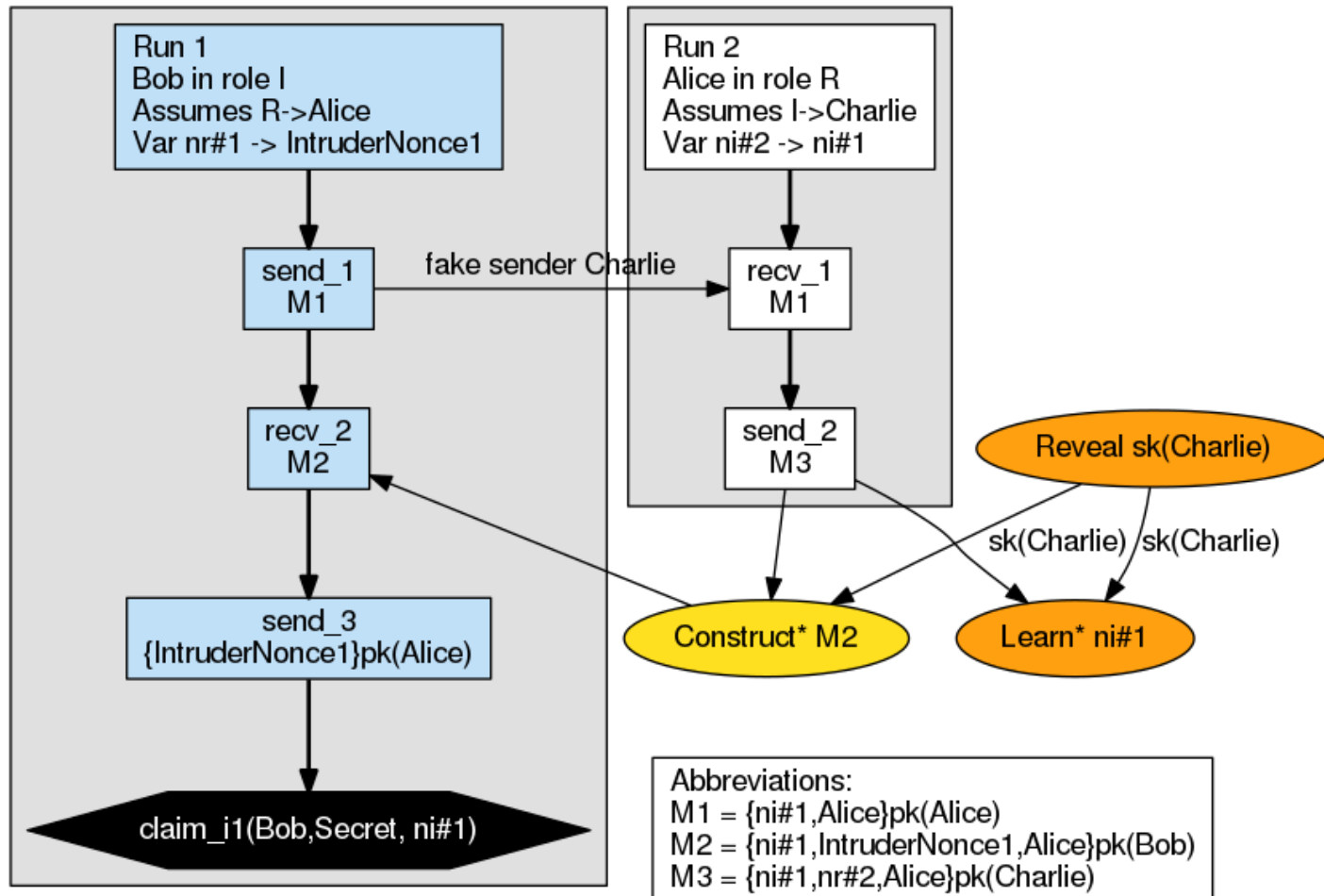
Scyther Tool

Scyther results : verify						
Claim				Status	Comments	Patterns
nsl3_broken	I	nsl3_broken,i1	Secret ni	Fail	Falsified	At least 1 attack.
		nsl3_broken,i2	Secret nr	Fail	Falsified	Exactly 1 attack.
		nsl3_broken,i3	Niagree	Fail	Falsified	Exactly 1 attack.
		nsl3_broken,i4	Nisynch	Fail	Falsified	Exactly 1 attack.
R		nsl3_broken,r1	Secret ni	Fail	Falsified	At least 1 attack.
		nsl3_broken,r2	Secret nr	Ok	Verified	No attacks.
		nsl3_broken,r3	Niagree	Ok	Verified	No attacks.
		nsl3_broken,r4	Nisynch	Ok	Verified	No attacks.

Scyther Tool

Attack i1

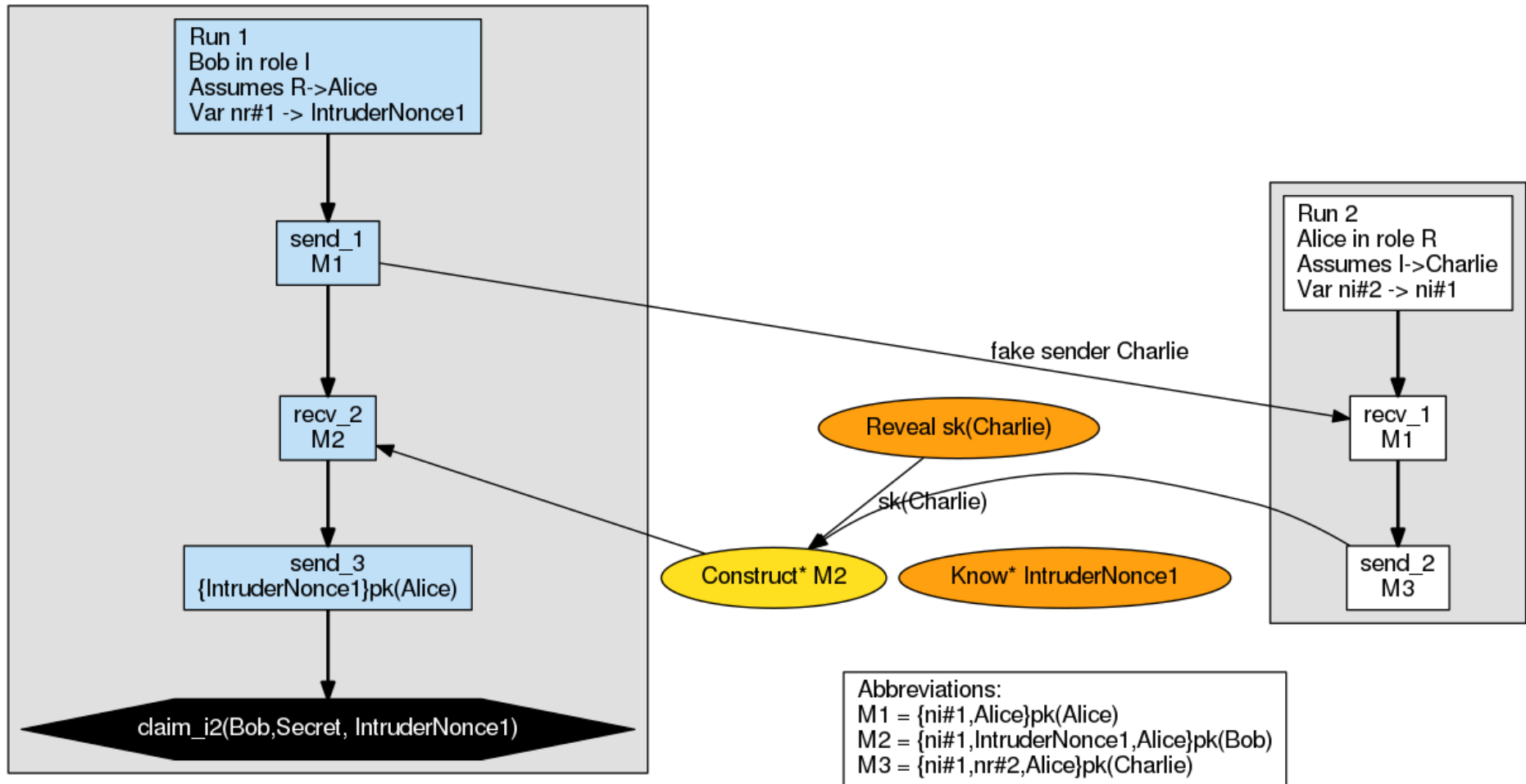
Attack for claim nsl3-broken,i1



Scyther pattern graph for the nsl3-broken protocol, claim nsl3-broken,i1 in role I.

Scyther Tool

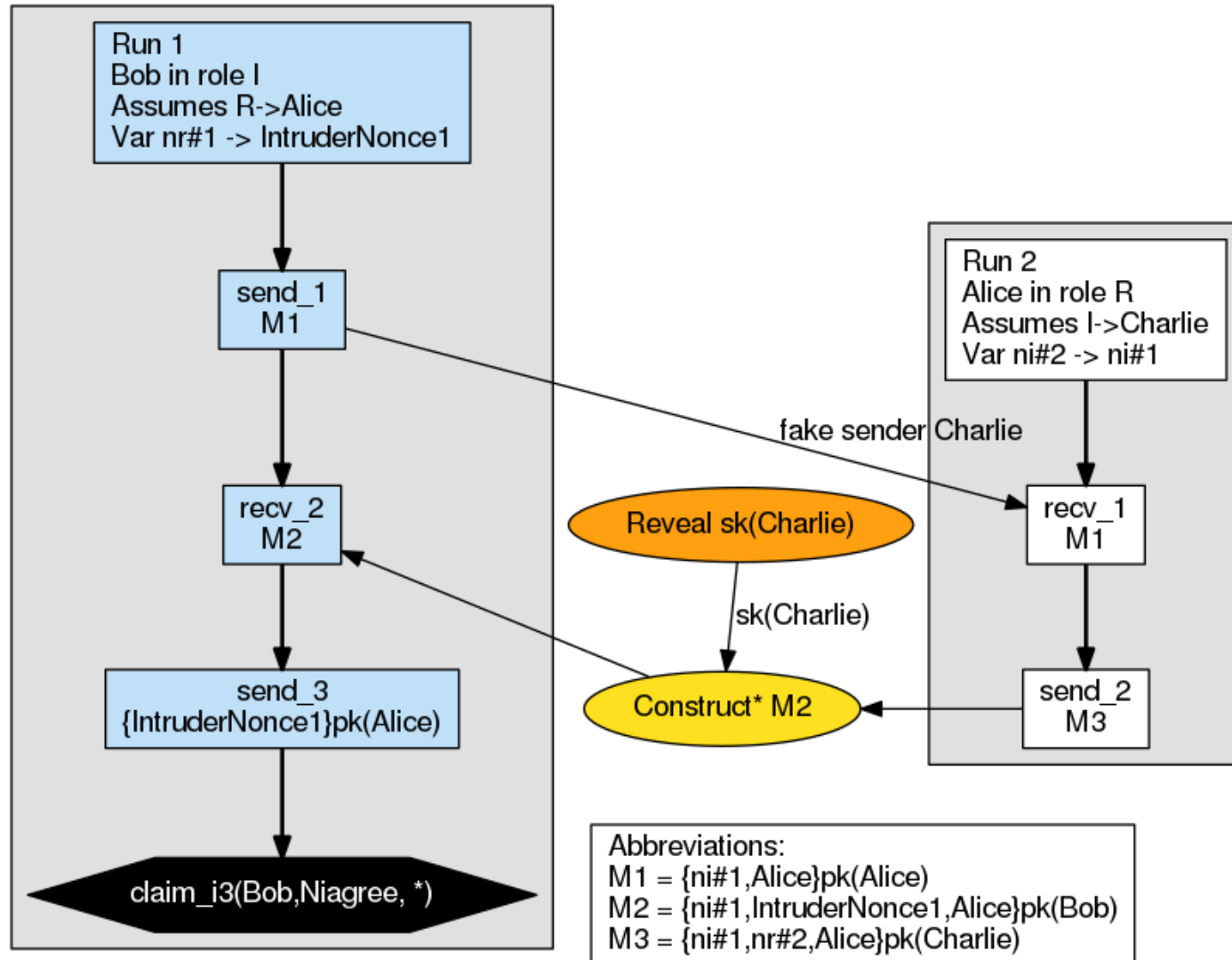
Attack i2



Scyther pattern graph for the ns13-broken protocol, claim ns13-broken,i2 in role I.

Scyther Tool

Attack i3



Scyther pattern graph for the ns13-broken protocol, claim ns13-broken,i3 in role I.